# ASSIGNMENT 5 - PART B

# B4 - FINAL ENGINEERING DECISION

Implementation Extension, Capacity Estimate and Production Recommendation | 5 Marks | K5 - Evaluate

## 1. Objective and Requirements

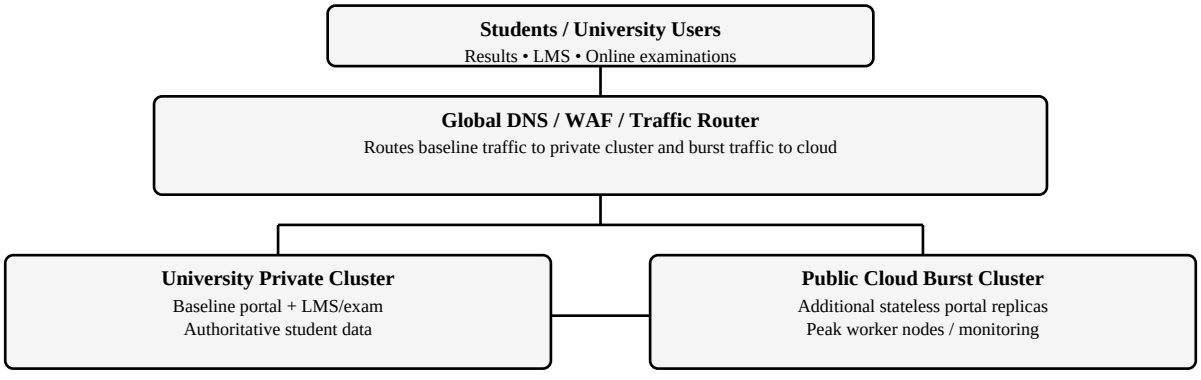
The university plans to host its learning-management system and online examinations on the same infrastructure as the result portal, while handling result day without purchasing new servers. B4 requires a second service with independent resource limits, a capacity estimate based on measured requests per second, and a production choice among own servers with VMs, own servers with containers/Kubernetes, public cloud or a hybrid model.

| Section | Marks | Required evidence |
|---|---|---|
| B4.1 - Practical Extension | 2 | A second service added to Kubernetes or Docker Compose; both services running with their own limits. |
| B4.2 - Capacity Estimate | 1 | Use measured requests/second to estimate containers or VMs for 150,000 students in two hours. |
| B4.3 - Production Decision | 2 | Select a deployment model and justify using measurements and cost estimates; evaluate public-cloud-only annual cost, scaling time and data control. |

The case study states 150,000 student logins in a two-hour window. The average login arrival rate is 150,000 / 7,200 = 20.83 logins per second. This is not necessarily the HTTP requests-per-second rate: one login can trigger several backend requests, and demand is not perfectly uniform. Therefore B4.2 must combine the B3 measured sustainable throughput with explicit request-per-login and burst assumptions.

## 2. B4.1 - Practical Extension: Add LMS / Online Examination Service

The B3 Kubernetes environment is extended with a separate `lms-exam` workload alongside `result-portal`. Each service has its own Deployment, Service, image, readiness endpoint and CPU/memory requests and limits. The demonstration LMS/exam application exposes a home page, `/health` and `/exam/status`; it demonstrates a second independently managed service and is not a complete examination platform.




| Service | Replicas | CPU request / limit | Memory request / limit | Purpose |
|---|---|---|---|---|
| Result Portal | 3 | 250m / 500m | 256Mi / 512Mi | Publishes semester results; scales separately. |
| LMS / Exam | 2 | 200m / 400m | 256Mi / 512Mi | Hosts the additional learning / examination service. |

### 2.1 Aggregate resource calculation

Initial portal request total: 3 × 250m = 750m CPU and 3 × 256Mi = 768Mi memory. Initial portal limits total: 3 × 500m = 1.5 CPU and 3 × 512Mi = 1.5Gi memory.

LMS/exam request total: 2 × 200m = 400m CPU and 2 × 256Mi = 512Mi memory. LMS/exam limits total: 2 × 400m = 800m CPU and 2 × 512Mi = 1Gi memory.

**Combined initial requests = 1.15 CPU and 1.25Gi memory. Combined limits = 2.3 CPU and 2.5Gi memory.**

The supplied namespace ResourceQuota permits up to 4 CPU / 5Gi of aggregate requests and 8 CPU / 10Gi of aggregate limits, with at most 15 Pods. These ceilings leave room for the result portal to scale while preventing unlimited consumption. Kubernetes schedules Pods using requests and enforces limits at run time; requests and limits should be tuned after workload measurement. [1][2]

## 2.2 Build, deploy and verify both services

```
# Build the two independent service images
docker build -t university/result-portal:b4 ./portal
docker build -t university/lms-exam:b4 ./lms_exam

# Load both images into the local KIND cluster
kind load docker-image university/result-portal:b4 --name b4
kind load docker-image university/lms-exam:b4 --name b4

# Apply both Deployments, Services and the namespace quota
kubectl apply -f kubernetes/b4-services.yaml
kubectl get deployments,pods,services -n b4-university
```

## 2.3 Test service health and individual resource settings

```
# Separate terminals: forward the portal and LMS/exam Services
kubectl port-forward -n b4-university service/result-portal 8000:80
curl http://127.0.0.1:8000/health

kubectl port-forward -n b4-university service/lms-exam 8001:80
curl http://127.0.0.1:8001/health
curl http://127.0.0.1:8001/exam/status
```

## Evidence interpretation

The result-portal Deployment should show three Ready Pods; the LMS/exam Deployment should show two Ready Pods. The two Services select separate labels, so requests are routed to the correct application.

Use `kubectl describe deployment` or `kubectl describe pod` to verify the different request and limit values. The namespace ResourceQuota provides an additional aggregate cap. `kubectl top pods` reports live usage when the Metrics API is installed; if it is unavailable, preserve the describe output and record the limitation.

The LMS/exam endpoint is a demonstration service used to prove independent deployment and resource control. It is not a production learning-management or online examination product. A real LMS/exam service needs authenticated users, secure exam sessions, durable storage, audit trails and its own performance/security testing.

# 3. B4.2 - Capacity Estimate from B3 Throughput

The calculator in `scripts/b4_capacity_estimator.py` takes the measured sustainable requests/second of one result-portal container from B3. Use the throughput at which the response-time and error-rate objectives are still met, not the highest rate observed while the service is overloaded.

## 3.1 Formula

Average login arrival rate: 150,000 / (2 × 60 × 60) = **20.83 logins/second**.

**Design RPS = 20.83 × requests per login × burst factor × headroom factor.**

**Required portal replicas = CEILING(Design RPS / measured sustainable RPS per container).**

The number of backend requests per login, peak-to-average burst factor and headroom must be measured or justified. The task does not provide them. They are explicit calculator inputs, not facts from the question.

## 3.2 Worked example - illustrative assumptions, not measured results

| Input | Illustrative value | Explanation |
|---|---|---|
| Students / window | 150,000 / 2 hours | Given by the assignment. |
| Backend requests per login | 5 | Planning assumption; replace using request traces. |
| Burst factor | 2.0 | Planning assumption to represent uneven arrivals. |
| Headroom factor | 1.25 | Planning reserve above estimated burst. |
| Measured sustainable RPS per one container | 40 RPS | Example only; replace with B3 measured throughput. |

Design RPS = 20.83 × 5 × 2.0 × 1.25 = **260.42 requests/second**. With an illustrative measured capacity of 40 sustainable requests/second per container, required portal replicas = CEILING(260.42 / 40) = **7 containers**.

At seven replicas, the result-portal workload requests 7 × 250m = 1.75 CPU and 7 × 256Mi = 1,792Mi (1.75Gi) of memory. Its configured limits total 3.5 CPU and 3.5Gi. Adding the two LMS/exam replicas gives combined requests of 2.15 CPU and 2.25Gi, and combined limits of 4.3 CPU and 4.5Gi. This fits the supplied namespace quota, but the actual cluster must have sufficient allocatable capacity and system headroom.

Use the actual B3 value to recompute this result. If measured per-container capacity differs, the replica count changes. The example must not be reported as the student's measured performance.

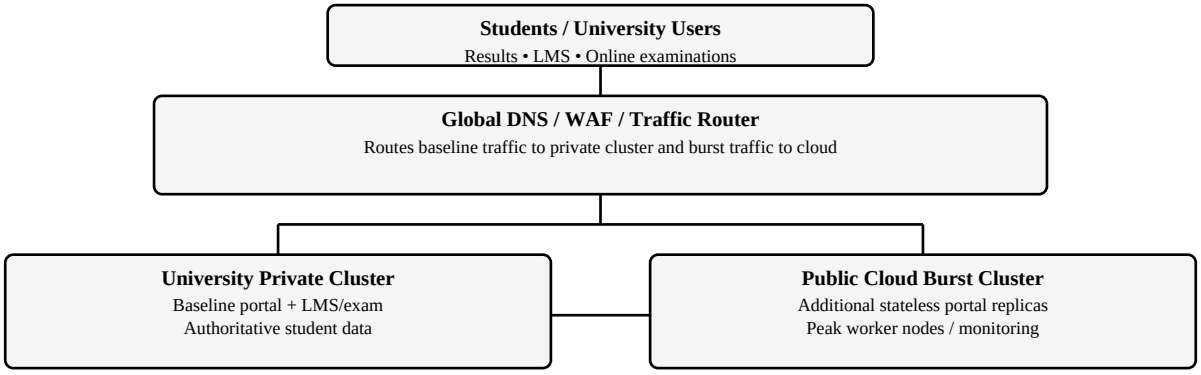
```
python3 scripts/b4_capacity_estimator.py \\
  --measured-rps YOUR_B3_MEASURED_RPS \\
  --requests-per-login YOUR_REQUESTS_PER_LOGIN \\
  --burst-factor YOUR_BURST_FACTOR \\
  --headroom 1.25
```

# 4. B4.3 - Production Deployment Decision

| Option | Benefits | Limitations / evaluation |
|---|---|---|
| Own servers with VMs | Separate guest OS/kernel per service; useful for legacy OS requirements and a clear guest boundary. | Guest OS memory, storage and boot overhead; capacity limited to existing hosts; one host failure can affect multiple VMs. |
| Own servers with containers + Kubernetes | Efficient workload density, resource limits, replica replacement and scheduling on existing nodes. | Peak capacity remains limited by existing server CPU/RAM; needs Kubernetes skills, monitoring, redundancy and spare capacity. |
| Public cloud only | Elastic worker capacity, hourly billing and managed database/load-balancer services; capacity can be added without buying university servers. | Provider hosts infrastructure and stores data; continuous database/control-plane/LB/storage and egress charges remain; identity, region, keys and contracts need governance. |
| Hybrid | Uses existing private infrastructure for baseline and data control, with public-cloud burst for stateless web replicas. | Requires secure networking, traffic routing, observability, data-replication design and tested cutover/failback. |

## 4.1 Recommended decision: hybrid

Select a hybrid architecture. Keep baseline services and the authoritative student-record database on the university's existing infrastructure, assuming B3 measurements confirm the private cluster can handle ordinary-day load. Before result release, add stateless portal replicas in a public-cloud burst environment and route traffic using a global load balancer or DNS-based traffic manager. Keep the LMS/exam service on the platform with capacity sized from its own workload; online examinations require separate load tests and strict isolation from the result-release spike.




Use two separately operated environments rather than assuming one Kubernetes control plane should stretch across an unreliable WAN. Connect private and cloud environments using encrypted private connectivity or a site-to-site VPN, service identities, restricted ports and health-based traffic routing. If read-only student results are replicated to the cloud, encrypt the transfer and storage, restrict the replica to the approved region, log access and test consistency/failback. Do not replicate sensitive examination data unless the university's data-governance approval explicitly permits it.

Data control: the university remains the owner and policy decision-maker for student records. In a public-cloud deployment the provider operates/hosts the infrastructure and processes data under the contract; it does not remove the university's responsibilities for identity, access, retention, residency and incident response. Use encryption in transit and at rest, university-controlled or customer-managed keys where supported, least-privilege roles, audit logging and contractual data-processing terms.

## 4.2 Capacity activation and data-protection sequence

1. Before the event, run the B3 capacity estimator using the measured sustainable requests/second per portal container and the actual request-per-login assumption.

2. Convert the replica count into node demand using each Pod's CPU and memory requests plus node allocatable capacity. Keep operating-system, system DaemonSet and failure headroom outside the application allocation.

3. Pre-scale the cloud burst environment before the published result time. Measure from the scale request until the extra worker nodes and portal Pods are Ready and the health endpoint responds. Record image-pull and readiness delays separately where possible.

4. Route traffic only to healthy private/cloud endpoints. Keep the LMS/exam service independently limited and avoid allowing examination workloads to consume the result portal's reserved capacity.

5. Keep the authoritative student database under the university's approved access and retention policy. Any cloud replica must use encrypted transport/storage, approved region placement, restricted roles, audit logs and tested consistency/failback.

This sequence makes the hybrid option conditional on the measured B3 capacity, verified private-network throughput and a successful pre-event scale test.

# 5. Alternative Concept - Public Cloud Only, Billed Hourly

One result day per semester means two high-traffic days in a year. If the stated two-hour window is the peak period, the simplified annual peak duration is 2 days × 2 hours = **4 peak hours/year**. Any pre-warming and cool-down capacity must be added separately.

## 5.1 Annual public-cloud cost model

A cloud-only Kubernetes model still has baseline infrastructure that runs all year: worker nodes for ordinary traffic, the managed Kubernetes control plane (where billed), the managed database, the external load balancer and persistent storage. Only the additional peak worker capacity is assumed to run for the four event hours in this simplified model.

$$C_{annual} = 8{,}760 \times (N_{base} \times P_{node} + P_{control} + P_{db} + P_{lb})$$
$$+ 4 \times (N_{peak} - N_{base}) \times P_{node}$$
$$+ \text{annual storage} + \text{backups} + \text{network egress} + \text{monitoring} + \text{support} + \text{taxes.}$$

| Variable | Meaning / evidence needed |
|---|---|
| N_base | Baseline worker-node count based on ordinary-day Pod requests and node allocatable capacity. |
| N_peak | Peak worker-node count based on B4 replica estimate and node allocatable capacity. |
| P_node | Hourly price per selected cloud worker node in the chosen region. |
| P_control | Hourly managed Kubernetes control-plane charge, if the selected service charges one. |
| P_db | Hourly managed database charge for the chosen database tier and HA mode. |
| P_lb | Hourly load-balancer base charge, excluding usage-based units where applicable. |
| Storage / backup | GB-month storage and retained backup volume × provider rates. |
| Egress / monitoring | Data transfer out, logs, metrics, alerts, support and tax. |

The task does not name a cloud provider, region, instance family, database tier, storage volume, egress volume or currency. A defensible rupee/dollar total therefore cannot be calculated from the assignment alone. Enter current rates from the selected provider's pricing calculator in `evidence/B4_capacity_cost_worksheet.txt`; include the quote date and currency. The worksheet gives every term in the annual formula so that no recurring service is omitted.

## 5.2 Time to add capacity

On existing nodes with spare allocatable resources, Kubernetes can schedule new Pods after the Deployment is scaled. If nodes are full, the cloud node autoscaler must provision additional worker nodes, then the image must be available and the Pods must pass readiness checks. The time depends on the cloud provider, node image, region capacity, registry/cache and workload start-up. Measure from the scale request until the new Pods are Ready; pre-scale before a known result-release time instead of relying on reactive scale-out at the first traffic spike.

## 5.3 Data ownership and control

Under cloud-only deployment, the university remains responsible for student-data governance and determines who can access records, what is retained and which region is approved. The cloud provider supplies and operates the contracted infrastructure and may process/store the records. Contracts, data-processing terms, encryption/key ownership, role-based access, audit logs, backup retention, export and deletion procedures must be specified before production.

## 6. Evidence and Final Decision

| Required evidence | What to attach / record |
|---|---|
| B4.1 extension | Both Deployments and Services, `kubectl get pods`, and `kubectl describe` output showing independent CPU/memory requests and limits. |
| B4.2 capacity | Actual B3 per-container sustainable RPS, workload assumptions, calculator output and final replica/resource total. |
| B4.3 cost | Provider, region, quote date, node/DB/LB/control-plane/storage/egress/monitoring rates and calculated annual total. |
| Production readiness | Measured time to add capacity, private connectivity plan, data-region/key policy, recovery and failback test. |

Final decision: use the university's existing infrastructure for baseline and authoritative data services, and burst stateless portal capacity into a public-cloud environment before result day. This avoids purchasing new servers while adding elastic peak capacity. Keep the LMS/online-exam workload separately resource-limited and load-tested. The decision is conditional on B3 proving that the current private infrastructure can sustain normal load and on the annual cloud-burst quote being lower than the cost/risk of provisioning permanent peak capacity.

## 7. Conclusion

B4 adds an independently limited LMS/online-examination service beside the result portal, estimates peak portal replicas from the actual B3 sustainable requests/second, and chooses a deployment model based on measured capacity and annual costs. The recommended plan is hybrid: retain university control of authoritative data and baseline services while using public cloud for event-driven stateless application capacity. Final replica count, annual currency total and time-to-capacity must be populated from the student's own benchmark and selected provider quote.

## 8. Technical References and Source Alignment

[1] Kubernetes Documentation, Manage Resources for Containers and Pods - requests and limits.
https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/

[2] Kubernetes Documentation, Resource Quotas - aggregate namespace limits for CPU, memory and Pods.
https://kubernetes.io/docs/concepts/policy/resource-quotas/

[3] Kubernetes Documentation, Assign CPU and Memory Resources - scheduling and runtime enforcement.
https://kubernetes.io/docs/tasks/configure-pod-container/assign-cpu-resource/